

Benchmarking Classical Machine Learning and Transfer Learning for Data-Constrained Jute Leaf Disease Classification

Christian Joseph Bunyi
De La Salle University
Manila, Philippines
christian_bunyi@dlsu.edu.ph

Immanuel Umali
De La Salle University
Manila, Philippines
immanuel_umali@dlsu.edu.ph

Abstract—Jute leaf disease classification can support automated, real-time detection via computer vision models deployed on edge devices. However, open-access jute leaf disease image datasets are often small or only cover a few diseases, making it challenging to train robust models. We unify three open jute leaf datasets into a 2,382-image, six-class dataset covering *Cercospora Leaf Spot*, *Dieback*, *General Damage*, *Healthy*, *Mosaic*, and *Stem Rot* diseases. We then use this dataset to benchmark five classical machine learning (ML) classifiers with raw-pixel and handcrafted-feature extractors, as well as several deep learning (DL) models fine-tuned via transfer learning. For classical ML, the tuned Random Forest model with handcrafted features achieved the best validation macro-F1 score of 77.73% and a test accuracy of 72.22%. For DL, the learning-rate-optimized MobileNetV2 achieved the best overall test accuracy of 91.39%. We also conducted diagnostic analyses using confusion matrices, feature-space projections, error inspection, feature importance, and Grad-CAM. Results suggest that *Mosaic* and *Cercospora Leaf Spot* remain difficult to distinguish due to visually overlapping symptoms and possible limitations of single-label preprocessing. Overall, fine-tuning standard deep learning models appears promising for data-constrained jute disease classification. Future work can explore more comprehensive image preprocessing and formulate the task as a multi-label classification problem.

Index Terms—Jute, plant disease classification, computer vision, machine learning, deep learning, transfer learning

I. INTRODUCTION AND RELATED WORKS

The jute plant, which is also locally referred to as *saluyot* in the Philippines, is a green leafy vegetable found across tropical regions from Asia to Africa. It is also referred to as a "golden fiber" in countries such as Bangladesh and India, where it acts as an eco-conscious substitute for plastic-based materials because it decomposes naturally without leaving harmful residues [1]. It is traditionally used for packing, carpet backing, weaving ropes, and bags. Modern applications revolve around its use in decorative materials including lampshades, wall covers, and curtains [2]. In 2024, the Philippines exported \$452k worth of jute, making it the 14th-largest exporter of jute and other textile fibers in the world [3]. Major producers such as China, Thailand, Myanmar, Indonesia, and Bangladesh greatly value this material, making it a critical agricultural and economic asset in Asia [2].

Jute plants are commonly affected by various diseases such as yellow mosaic and *Cercospora* leaf spot. Traditionally, the recognition process relies on manual identification by farmers and agricultural experts. However, this approach is limited by its time-consuming nature and its need for reliable disease knowledge, which makes it impractical and often unavailable for large-scale farming situations. Furthermore, visual inspection is subjective and prone to human error, leading to inconsistent and potentially risky results [4].

Recent advancements in computer vision have led to the potential use of machine learning and deep learning technologies to classify these diseases. A prior study [5] developed an Inception V3 model that accurately classified jute leaves with powdery mildew disease and yellow mosaic. Their model achieved outstanding results, reaching an accuracy of 99.98%; however, their study was limited by the small number of diseases classified. On the other hand, another study [1] targeted a different set of diseases, specifically *Cercospora Leaf Spot* and *Golden Mosaic*, which are characterized by brown lesions and yellow patches, respectively. Their study developed an explainable convolutional neural network (CNN) through a supervised and semi-supervised self-training approach. The model achieved mean accuracy, precision, recall, and F1-score values that were all above 98%, demonstrating strong stability while reducing reliance on costly labeled datasets, considering its 97.8% mean accuracy during the semi-supervised test. In contrast to the aforementioned works, a federated CNN approach [6] was proposed to classify five types of jute leaf diseases and achieved classification accuracies ranging from 79.87% to 83.67%. While numerous studies demonstrate strong potential for classifying jute leaf images, such studies are limited by the small number of disease classes considered or by high computational overhead [7]–[9]. To address this, this study explores classical machine learning and deep learning methods for jute leaf disease classification.

We summarize our main contributions as follows:

- 1) We unify three open datasets on jute leaf diseases.
- 2) We benchmark classical ML models and several deep-learning backbones to test whether a compact architecture such as MobileNetV2 can remain competitive with

TABLE I
SUMMARY OF CHOSEN JUTE LEAF DISEASE DATASETS

Source	Environment	Samples	Class Mapping
Islam & Sheikh [10]	Field	1,390	Dieback → Dieback; Fresh → Healthy; Holed → General Damage; Mosaic → Mosaic; Stem Soft Rot → Stem Rot
Mridha [11]	Controlled	72	Jute Plant Leaves → Healthy
Jannat et al. [1]	Field, Controlled	920	Cercospora Leaf Spot → Cercospora Leaf Spot; Golden Mosaic → Mosaic; Healthy Leaf → Healthy
Unified Dataset	Field, Controlled	2,382	Cercospora Leaf Spot, Dieback, General Damage, Healthy, Mosaic, Stem Rot

TABLE II
CLASS DISTRIBUTION OF THE UNIFIED JUTE LEAF DISEASE DATASET

Class	Count	Share
Cercospora Leaf Spot	309	12.97%
Dieback	300	12.59%
General Damage	300	12.59%
Healthy	616	25.86%
Mosaic	587	24.64%
Stem Rot	270	11.34%
Total	2,382	—

larger backbones on a small merged jute-leaf dataset.

- 3) We conduct domain-pretraining, transfer-learning, input-resolution, and learning-rate optimization experiments for MobileNetV2.

II. DATASETS

We gathered several jute leaf disease datasets online and unified them into a single dataset to improve the generalizability of our study. We first came across a data journal paper on an image dataset of jute leaf diseases by Islam and Sheikh, accessible through Harvard Dataverse [10]. Their dataset contained 1,390 images categorized into five classes, which was small for an image recognition task, so we searched for other similar datasets. We eventually found two similar open-access datasets from Mendeley [11] and Kaggle [1] that could be merged with the first, resulting in a single dataset containing 2,382 images categorized into six classes after preprocessing: Cercospora Leaf Spot, Dieback, General Damage, Healthy, Mosaic, and Stem Rot. We considered several other similar datasets, but decided not to include them because the authors did not respond to our data request or the data was unlabeled.

Since our unified jute leaf disease dataset was still relatively small for an image recognition task, we experimented with domain pretraining on two related plant leaf disease datasets:

- 1) PlantVillage: A dataset containing 54,305 images across 14 crop species and 38 classes of healthy and diseased crop leaves [12].
- 2) PlantDoc: A dataset containing 2,598 images across 13 plant species and 17 disease categories [13]; in the downloaded Kaggle folder structure used for domain pretraining, we treated it as a flat 27-class image classification task representing distinct plant-condition pairs.

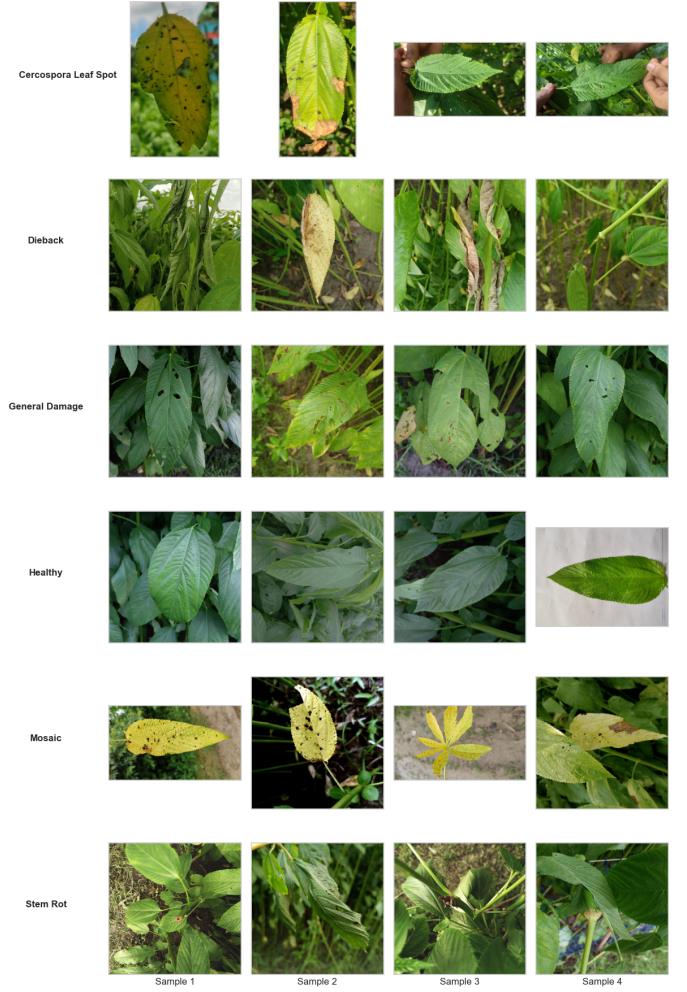


Fig. 1. Sample images from the unified jute leaf disease dataset.

III. METHODOLOGY

A. Experimental Setup

We implemented the project using Python 3.11. Classical ML experiments were conducted on Google Colab's CPU with 12.7 GB of system RAM, while DL experiments were conducted on Google Colab's L4 GPU with 53.0 GB of system RAM and 22.5 GB of GPU RAM. PyTorch-based training used pre-trained backbones integrated through *timm*; standard Python image-processing and ML libraries supported augmentation, feature extraction, feature-space analysis, visualization, and Grad-CAM interpretability.

B. Data Preprocessing

We curated the unified jute leaf disease dataset described in the Datasets section by unifying labels into six classes (Tables I and II). The dataset was partitioned offline before feature extraction or preprocessing using a stratified 70%/15%/15% train/validation/test split with a default random seed of 42. Images were shuffled within each class folder independently, after which the first 70% were assigned to training, the next 15% to validation, and the remaining images to testing. This yielded 1,666 training images, 356 validation images, and 360

test images (Table III). We did not perform content-based duplicate or near-duplicate removal using perceptual hashing or cryptographic hashes; consequently, residual near-duplicate or source-scene leakage remains a limitation. Nevertheless, the split was fixed before all experiments, validation data were used for model selection, and the isolated test set was used only for final evaluation.

TABLE III
PER-CLASS TRAIN/VALIDATION/TEST SPLIT COUNTS

Class	Train	Val.	Test	Total
Cercospora Leaf Spot	216	46	47	309
Dieback	210	45	45	300
General Damage	210	45	45	300
Healthy	431	92	93	616
Mosaic	410	88	89	587
Stem Rot	189	40	41	270
Total	1,666	356	360	2,382

To address class imbalance and reduce overfitting, we implemented weighted random sampling and applied stochastic data augmentation during training. For reproducibility, all stochastic operations were seeded using a fixed integer. These transformations were applied in the following sequence:

- `RandomResizedCrop` was applied to standardize images to 256×256 pixels.
- `SquareSymmetry` ($p = 0.5$), and `Affine` transforms ($\theta = \pm 15^\circ$, shear $\pm 15^\circ$) were applied to simulate different perspectives.
- `CoarseDropout` (1–8 holes, size 5–10% of dimensions, $p = 0.2$) was applied to occlude leaf portions, encouraging the model to rely on global features rather than localized features.
- `RandomBrightnessContrast` and `PlanckianJitter` (effective $p = 0.4$) were applied to simulate field lighting variations and outdoor color temperature shifts.
- `GaussNoise`, `ISONoise`, and environmental blurs (`GaussianBlur` and `MotionBlur` with 3×3 to 5×5 kernels, $p = 0.2$) were applied to simulate image quality variations.

Finally, we normalized all deep learning (DL) inputs with ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$) to align input distributions with pre-trained backbones. We omitted this normalization for classical ML models so that handcrafted feature extractors would operate on unshifted color and texture distributions.

For ML feature extraction, we used two extractors: (1) raw pixel feature extractor (RPFE) and (2) crafted feature extractor (CFE). The RPFE takes the standardized 256×256 input image and flattens the RGB values into a 196,608-dimensional vector, which is then passed to the ML training pipeline. The CFE used the following features, which were selected based on [14]:

- Hue, Value, and Saturation (HSV): Extracted six color moments (mean and standard deviation of each channel) and a 512-bin quantization ($8 \times 8 \times 8$)

- Local Binary Patterns (LBP): Uniform LBP ($r = 3$, $points = 24$) were calculated to characterize the micro-texture of the leaf surface.
- Gray-Level Co-occurrence Matrix (GLCM): Generated across multiple distances ($d = 1, 3$) and orientations ($\theta = 0^\circ, 45^\circ, 90^\circ, 135^\circ$). Six statistical properties were computed—Contrast, Dissimilarity, Homogeneity, Energy, Correlation, and Angular Second Moment (ASM).
- Gabor Filters: Applied four kernels with orientations $0, \frac{\pi}{4}, \frac{\pi}{2}$, and $\frac{3\pi}{4}$ to capture directional textures.
- Histogram of Oriented Gradients (HOG): Analyzed gradient orientations within 16×16 pixel cells and 2×2 cell blocks.

For classical ML models, features were extracted separately for the training, validation, and test splits. When scaling was required, normalization statistics were learned only from the training split and applied to validation and test splits through a scikit-learn `Pipeline`. The balanced classical-ML setting computed training-split sample weights as $w_c = \frac{N}{C \times n_c}$ and supplied these weights to supported estimators during fitting. For DL models, pre-split directories were loaded using `ImageFolder`; training loaders could use a `WeightedRandomSampler` with class weights computed from the active training labels, while validation and test loaders used deterministic transforms only. Although the implementation supported cross-validation by merging the training and validation splits into an 85% development pool and repartitioning it using stratified folds, the test split remained untouched.

For validation and testing, we used only `SmallestMaxSize` rescaling and center-cropping (at 256×256 or 512×512 resolution) to ensure fair evaluation.

C. Evaluation

We evaluated the models using accuracy and macro-averaged precision, recall, and F1-score (macro-F1). A macro-averaged metric is first computed per class, then averaged across all classes, while a micro-averaged metric is computed across all samples. Effectively, macro-averaging treats each class equally while micro-averaging treats each sample equally. For DL experiments, the best checkpoint was selected using validation loss, and the isolated test loader was evaluated through the final test routine only after model selection.

IV. RESULTS AND DISCUSSION

A. Performance Analysis

We compared the performance of classical ML and deep learning models to identify the most effective architecture for jute disease classification.

1) *ML Model Benchmarking*: We benchmarked five classical classifiers (K-Nearest Neighbors, Gaussian Naive Bayes, Logistic Regression, Random Forest (RF), and Support Vector Machine (SVM)) with two feature extraction strategies, RPFE and CFE, for a total of 10 classical ML pipeline configurations. Before hyperparameter tuning, RF-CFE achieved the highest validation accuracy of 76.97% and validation macro-F1 of 74.69%, while SVM-CFE achieved the highest test accuracy

of 70.22% and test macro-F1 of 71.73% among the baseline classical ML pipelines.

2) *DL Architecture Benchmarking*: We benchmarked EfficientNet, ResNet, Inception, MobileViT, and MobileNetV2 architectures [15]–[19]. Transfer learning was conducted on all architectures using the pre-trained weights listed in Table IV with default training hyperparameters. Our results show that EfficientNet-B5 achieved the highest validation accuracy of 90.44%, followed by MobileNetV2 with 89.04% as summarized in Table IV. Interestingly, despite EfficientNet-B7 having the most parameters, it performed the worst among the architectures. Moving forward, we decided to focus on optimizing the MobileNetV2 architecture, as it was small yet performant, making it a practical choice.

The MobileNetV2 architecture-baseline training history showed that training metrics lagged behind their validation counterparts, which can be explained by data augmentation and regularization techniques that made training more difficult. The LR-optimized MobileNetV2 training history was more erratic due to the increased learning rate of $\eta = 0.01$.

TABLE IV
DEEP LEARNING ARCHITECTURE COMPARISON

Architecture	HuggingFace Model Name	Params	Val. Acc.	Test Acc.
EfficientNet-B7 [15]	timm/tf_efficientnet_b7.ns_jft_in1k	66.3M	78.37%	78.61%
EfficientNet-B5 [15]	timm/efficientnet_b5.sw_in12k_ft_in1k	30.4M	90.44%	88.61%
ResNet-50 [16]	timm/resnet50.al_in1k	25.6M	83.14%	81.94%
Inception v3 [17]	timm/inception_v3.tv_in1k	23.8M	83.71%	84.72%
MobileViT (small) [18]	timm/mobilevit_s.cvnets_in1k	5.6M	87.36%	88.33%
MobileNetV2 [19]	timm/mobilenetv2_100.ra_in1k	3.5M	89.04%	90.27%

3) *Hyperparameter Grid Search*: ML optimizations were performed on RF-CFE and SVM-CFE separately. For RF-CFE, grid search was used to tune the hyperparameters. The hyperparameters for the grid search are defined in Table V, where underlined values represent the default hyperparameters used during baseline training. For SVM-CFE, randomized search was used with the hyperparameters defined in Table VI, which follows a similar format to Table V. The optimal values indicated in Tables V and VI yielded validation macro-F1 scores of 77.73% for RF-CFE and 73.40% for SVM-CFE; the tuned RF-CFE model achieved 72.22% test accuracy.

TABLE V
RANDOM FOREST GRID SEARCH HYPERPARAMETERS

Hyperparameter	Search Space	Optimal Value
n_estimators	{100, 200, 500}	200
max_depth	{None, 15, 25}	None
min_samples_leaf	{2, 5, 10}	2
max_features	{sqrt, log2}	sqrt

On the other hand, the first round of DL optimizations focused on conducting a grid search on three domain-pretraining

TABLE VI
SVM RANDOMIZED SEARCH HYPERPARAMETERS

Hyperparameter	Search Space / Distribution	Optimal Value
C (Regularization)	Log-Uniform [$\underline{1}$, 10^3]	3.5514
γ (Gamma)	{‘scale’, 0.1, 0.01, 0.001}	scale
Kernel	{‘rbf’, ‘poly’}	rbf

levels with transfer learning and three dropout rates $D \in \{0.0, 0.1, 0.2\}$. We defined three domain-pretraining levels based on the degree of domain-specific fine-tuning performed on the backbone prior to the final transfer learning stage on our unified jute dataset:

- Level 1 (L1): Transfer learning was performed directly from standard ImageNet-1K pre-trained weights.
- Level 2 (L2): The backbone was first fine-tuned on PlantVillage (38 classes), then transferred to jute.
- Level 3 (L3): Starting from L2 weights, the backbone was further fine-tuned on PlantDoc formatted as a flat 27-class plant-condition classification task, then transferred to jute.

During the transfer learning stages, the backbone was frozen, and only the newly initialized classifier head was trained using the AdamW optimizer [20] with the hyperparameters summarized in Table VIII. As seen in Table VII, L1 with a 0.0 dropout rate achieved the highest validation accuracy and validation macro-F1, outperforming its higher-domain-pretraining-level counterparts. However, at the same domain-pretraining level, dropout rates of 0.1 and 0.2 were not much lower, and we did not perform k-fold cross-validation, so we cannot rigorously conclude that a 0.0 dropout rate is the best. Furthermore, we chose to optimize L1 initialization with a dropout rate of 0.1 rather than 0.0, because enforcing some regularization may improve generalization.

These results contradicted our hypothesis that additional domain pretraining would improve model performance, suggesting that the general-purpose features learned from ImageNet were more adaptable to our task than the specialized but potentially distracting features learned from related plant leaf datasets.

TABLE VII
GRID SEARCH MATRIX ACROSS FINE-TUNING LEVELS AND DROPOUT RATES (VAL. ACC. / VAL. F1 / TEST ACC., IN %)

Level	$D = 0.0$	$D = 0.1$	$D = 0.2$
L1	89.33 / 89.82 / 88.06	89.04 / 89.55 / 88.33	89.04 / 89.55 / 87.22
L2	82.30 / 82.31 / 80.00	82.02 / 82.09 / 80.28	81.46 / 81.62 / 79.17
L3	83.43 / 83.58 / 83.33	83.15 / 83.41 / 83.61	84.83 / 84.20 / 83.06

TABLE VIII
COMMON TRAINING HYPERPARAMETERS FOR MOBILENETV2

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	0.001
Weight Decay	0.01
Batch Size	32
Max Epochs	50
Early Stopping Patience	5

4) *Input Resolution Analysis (DL)*: We were curious about how training the model on higher-resolution images would affect its performance, so we trained MobileNetV2 models on 512×512 -pixel images with dropout rates of 0.0 and 0.1 and compared them with their 256×256 -pixel counterparts. The models trained on 512×512 -pixel images underperformed their 256×256 -pixel counterparts (see Table IX).

TABLE IX
RESOLUTION-DROPOUT MATRIX FOR MOBILENETV2 (VAL. ACC. / VAL. F1 / TEST ACC., IN %)

Resolution	$D = 0.0$	$D = 0.1$
256×256	89.33 / 89.82 / 88.06	89.04 / 89.55 / 88.33
512×512	85.67 / 86.08 / 87.78	85.67 / 86.08 / 87.78

B. Diagnostic Analysis

We analyzed the baseline and tuned RF-CFE, SVM-CFE, and MobileNetV2 models and identified persistent confusion between *Mosaic* and *Cercospora Leaf Spot*. For RF-CFE and SVM-CFE, we tested whether maximum-depth control and the L2-regularization constant C , respectively, would improve accuracy and macro-F1. As shown by the optimal configuration in Table V, leaving maximum depth unconstrained yielded the best RF-CFE macro-F1. The second-best RF-CFE configuration used the same values except for setting maximum depth to 25 and yielded a validation macro-F1 of 76.67%. This small gap suggests that a relatively complex decision boundary may be useful for distinguishing the disease classes, although the unconstrained and depth-25 configurations performed similarly. For SVM-CFE, validation macro-F1 improved to 73.40% after C increased from 1.0000 to 3.5514. The second-best configurations shared the same values except for C , with three tied values ($C = 13.2929$, $C = 157.0297$, and $C = 19.7622$), suggesting that SVM-CFE performance plateaued once C exceeded a relatively low value.

As for MobileNetV2, we wanted to see if this could be resolved by trying different learning rates $\eta \in \{0.01, 0.005, 0.001, 0.0005, 0.0001\}$ and increasing the early-stopping patience to 20 epochs. Although the best learning rate ($\eta = 0.01$) achieved a test accuracy of 91.39% (see Table X), the confusion matrix comparison showed that the model remained confused in distinguishing between *Mosaic* and *Cercospora Leaf Spot*. This confusion may be caused by the visual similarities between these two diseases and a preprocessing flaw that optimization alone cannot resolve. As summarized in Table X, reducing the learning rate below the baseline value of $\eta = 0.001$ resulted in lower classification performance and significantly longer training times, with $\eta = 0.0001$ needing 49 epochs to achieve the lowest test accuracy of only 88.61%.

We then analyzed and compared the top 10 most confident errors for the baseline and tuned RF-CFE and SVM-CFE models, as well as the architecture-baseline and LR-optimized MobileNetV2 models. Indeed, the majority of misclassifications involved the *Mosaic-Cercospora Leaf Spot* pair. For MobileNetV2, this mostly occurred when the model prioritized localized spot textures over global leaf yellowing. Some

TABLE X
IMPACT OF LEARNING RATE ON MOBILENETV2 PERFORMANCE

Learning Rate (η)	Val. Acc.	Val. F1	Test Acc.
0.01	88.76%	89.11%	91.39%
0.005	88.20%	89.02%	90.00%
0.001 (Baseline)	87.64%	88.10%	90.28%
0.0005	85.67%	86.44%	88.89%
0.0001	83.71%	84.55%	88.61%

misclassifications in the *Dieback* class were attributed to cropped images that excluded withered regions. Finally, some misclassifications in the *General Damage* class were caused by field backgrounds filled with several jute leaves.

To better understand the low-dimensional structure of the high-dimensional jute leaf data, we projected the feature space using two nonlinear dimensionality reduction techniques: t-SNE and UMAP. For the handcrafted features, the classes did not form clearly separable clusters; however, *Dieback*, *General Damage*, and *Healthy* appeared more compact, while *Cercospora Leaf Spot*, *Mosaic*, and *Stem Rot* were sparsely distributed and overlapping. Because the handcrafted features include HSV color descriptors, this overlap may partly reflect color-based symptom similarities, such as brown lesions in *Cercospora Leaf Spot*, yellow patches in *Mosaic*, and dark brown spots in *Stem Rot*. In contrast, the DL projections formed more distinct clusters for *Stem Rot*, *General Damage*, and *Healthy*, but the *Mosaic* and *Cercospora Leaf Spot* clusters still partially overlapped. This overlap remained consistent across varying t-SNE perplexities and UMAP neighborhood values, suggesting that the model perceives these classes as highly similar.

C. Interpretability

For the classical machine learning models, we generated feature importance charts for RF-CFE and SVM-CFE using Gini importance and grouped permutation importance, respectively.

For RF-CFE, the strongest classical ML model, feature importance indicated that the model primarily depended on HOG features. Color moments, particularly the standard deviation of the green channel (σ_{green}) and the mean of the red channel (μ_{red}), also contributed to the model's decisions. These features may help distinguish symptoms such as brown lesions in *Cercospora Leaf Spot*, yellow patches in *Mosaic*, and dark brown spots in *Stem Rot*.

Lastly, for MobileNetV2, we generated Grad-CAM heatmaps to analyze the physical leaf sections the model focused on during classification. The model attended to relevant pathological features across most disease classes, such as necrotic stem regions in *Dieback* and physical tears in *General Damage*. However, some examples of the *Healthy* class showed that the model sometimes attended to clean tabletop backgrounds rather than the jute leaves themselves. This behavior suggests that the model may have learned to associate clean backgrounds with healthy samples. For the *Mosaic* class, we found that the model attended to localized

leaf spots over global yellowing/goldening, which explains its persistent confusion with *Cercospora Leaf Spot*.

Similar attention patterns were also observed in the LR-optimized model, further suggesting that its performance is constrained by the data and its preprocessing.

V. CONCLUSION

Our work on jute disease image classification using classical machine learning and deep learning allowed us to explore different strategies for classifying *Cercospora Leaf Spot*, *Dieback*, *General Damage*, *Healthy*, *Mosaic*, and *Stem Rot* in jute leaves. Our training, error analysis, and fine-tuning process showed that the LR-optimized MobileNetV2 model stood out among the evaluated models. The error analysis also provided insights that may improve future model performance. First, the images may require additional preprocessing, because the Grad-CAM heatmaps show instances where the deep learning model focuses on irrelevant subjects, such as background leaves, rather than the target leaf. Next, both classical ML and DL models struggled to distinguish *Cercospora Leaf Spot* from *Mosaic*. Inspection of these misclassified images suggests that some leaves may exhibit multiple disease symptoms, which exposes a limitation of the single-label formulation used in this study. This finding points to future work on stronger leaf/background preprocessing and a multi-label learning framework rather than a standard softmax classifier.

REFERENCES

- [1] M. Jannat, M. S. Uddin, M. A. Hasan, M. S. Alam, A. Paul, M. E. H. Chowdhury, and J. Haider, "Real-time jute leaf disease classification using an explainable lightweight CNN via a supervised and semi-supervised self-training approach," *Frontiers in Plant Science*, vol. 16, Oct. 2025.
- [2] B. S. Mahapatra, S. Mitra, T. Ramasubramanian, and M. K. Sinha, "Research on jute (*Corchorus olitorius* and *C. capsularis*) and kenaf (*Hibiscus cannabinus* and *H. sabdariffa*): present status and future perspective," *The Indian Journal of Agricultural Sciences*, vol. 79, no. 12, pp. 951–967, 2009.
- [3] [Online]. Available: <https://oec.world/en/profile/bilateral-product/jute-and-other-textile-fibers/reporter/phl>
- [4] M. S. Uddin and M. Munsir, *JuLeDI: Jute Leaf Disease Identification Using Convolutional Neural Network*. Springer, 10 2023, pp. 661–677.
- [5] R. Haque, M. M. Miah, S. Sultana, H. Fardin, A. A. Noman, A. Al-Sakib, M. K. Hasan, A. Rafy, R. M. Shihabur, and S. Rahman, "Advancements in Jute Leaf Disease Detection: A Comprehensive Study Utilizing Machine Learning and Deep Learning Techniques," in *2024 IEEE International Conference on Power, Electrical, Electronics and Industrial Applications (PEEIACON)*, Sep. 2024, pp. 248–253.
- [6] A. Bansal, S. Vats, C. Prasad, V. Kukreja, and S. Mehta, "Collaborative computing: federated learning cnns for jute leaf disease analysis," in *2024 3rd International Conference for Innovation in Technology (INOCON)*. IEEE, 2024, pp. 1–6.
- [7] D. Li, F. Ahmed, N. Wu, and A. I. Sethi, "Yolo-jd: A deep learning network for jute diseases and pests detection from images," *Plants*, vol. 11, no. 7, Mar. 2022. [Online]. Available: <https://www.mdpi.com/2223-7747/11/7/937>
- [8] Z. N. Reza, F. Nuzhat, N. A. Mahsa, and M. H. Ali, "Detecting jute plant disease using image processing and machine learning," *2016 3rd International Conference on Electrical Engineering and Information Communication Technology (ICEEICT)*, p. 1–6, Sep. 2016.
- [9] M. Z. Hasan, M. S. Ahamed, A. Rakshit, and K. M. Z. Hasan, "Recognition of jute diseases by leaf image classification using convolutional neural network," *2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, p. 1–5, Jul. 2019.
- [10] M. M. Islam and M. R. Sheikh, "A comprehensive image dataset of jute diseases," *Data in Brief*, vol. 64, p. 112334, Feb. 2026.
- [11] M. H. Mridha, "Jute Plant Leaves," May 2024.
- [12] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, Sep. 2016.
- [13] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A Dataset for Visual Plant Disease Detection," in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, ser. CoDS COMAD 2020. New York, NY, USA: Association for Computing Machinery, Jan. 2020, pp. 249–253.
- [14] S. M. Javidan, A. Banakar, K. Rahnama, K. A. Vakilian, and Y. Amptzidis, "Feature engineering to identify plant diseases using image processing and artificial intelligence: A comprehensive review," *Smart Agricultural Technology*, vol. 8, p. 100480, Aug. 2024.
- [15] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [17] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
- [18] S. Mehta and M. Rastegari, "Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer," *arXiv preprint arXiv:2110.02178*, 2021.
- [19] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.